CPSC 250
Derived Classes and Default Values

Edward Brash

1 Motivation

Object-oriented programming becomes much more powerful when we can create new classes from existing classes.

This is called **inheritance**.

A derived class inherits variables and methods from a base class, then adds more specific behavior.

2 Base Class: Transportation

Suppose we begin with a general class representing a transportation mode.

```
class Transportation:

    def __init__(self, name, speed=75):
        self.name = name
        self.speed = speed

    def info(self):
        print(f"{self.name} can go {self.speed} mph")
```

The default speed is:

75 mph

This means that if the user does not specify a speed, Python uses 75 automatically.

3 Why Use a Base Class?

There are many modes of transportation:

- motorized vehicles,
- self-powered vehicles,
- flight vehicles,
- water vehicles,
- and many others.

All of these are transportation modes, so it is useful to put common features in a shared base class.

4 Derived Class: MotorizedVehicle

A motorized vehicle is a transportation mode, but it has additional properties.

For example, it has:

- miles per gallon,
- fuel level,
- ability to add fuel,
- ability to drive.

```
class MotorizedVehicle(Transportation):

    def __init__(self, name, speed=80, mpg=40):
        Transportation.__init__(self, name, speed)

        self.mpg = mpg
        self.fuel_gal = 0

    def add_fuel(self, amount):
        self.fuel_gal += amount

    def drive(self, distance):
        required_fuel = distance / self.mpg

        if self.fuel_gal < required_fuel:
            print("Not enough fuel")

        else:
            self.fuel_gal -= required_fuel
            print(f"{self.fuel_gal} gallons remain")
```

5 Explicitly Calling the Base Constructor

The line:

```
Transportation.__init__(self, name, speed)
```

explicitly calls the constructor of the base class.

This initializes the part of the object inherited from `Transportation`.

This is extremely important: the derived class constructor does not automatically run the base class constructor unless we ask it to.

6 Derived Class: Motorcycle

A motorcycle is a specific kind of motorized vehicle.

```
class Motorcycle(MotorizedVehicle):

    def __init__(self, name, speed=55, mpg=25):
        MotorizedVehicle.__init__(self, name, speed, mpg)
```

```
    def wheelie(self):
        print("That's dangerous!")
```

The `Motorcycle` class inherits:

- `name`,
- `speed`,
- `mpg`,
- `fuel_gal`,
- `info()`,
- `add_fuel()`,
- `drive()`.

It also adds:

```
wheelie()
```

7 Using the Motorcycle Class

```
scooter = Motorcycle("Vespa", 55, 40)
dirtbike = Motorcycle("KX450F", 80, 25)

scooter.info()
dirtbike.info()
```

We can also rely on default values:

```
harley = Motorcycle("Harley Davidson")

harley.info()
print(harley.mpg)
print(harley.fuel_gal)
```

The object `harley` is created using the default values from the `Motorcycle` constructor.

8 A Generic Motorized Vehicle

We can still instantiate the parent class directly:

```
generic = MotorizedVehicle("People Carrier")

generic.info()
print(generic.mpg)
print(generic.fuel_gal)
```

This is useful when the base or intermediate class is meaningful on its own.

9 Inheritance Chain

The inheritance chain in this example is:

$$\texttt{Transportation} \rightarrow \texttt{MotorizedVehicle} \rightarrow \texttt{Motorcycle}$$

Each derived class inherits everything from the classes above it, unless behavior is overridden or hidden.

10 Key Takeaways

- A derived class inherits from a base class.
- Default values make constructors more flexible.
- Derived constructors should initialize the base part of the object.
- In Python, this is often done explicitly with a base-class constructor call.
- Inheritance allows us to reuse common code while adding specialized behavior.